

Risk Assessment of Insurance Policies using Machine Learning Techniques

Lukáš Trstenský, Bartosz Kochanski and Maia Ghionea

<https://github.com/SAKULAK/ITU-MachineLearning-FinalProject-Couriers>

Contents

1	Introduction	2
1.1	Machine Learning Methods	2
2	Data	2
2.1	Data Format	2
2.2	Data Cleaning	2
2.3	Visualization	2
3	Features	2
3.1	Parameters	3
3.2	Clustering	3
3.3	PCA	3
4	M1 - Decision Tree	3
5	M2 - Feed-Forward Neural Network	3
5.1	The Structure	3
5.2	The Flow	3
5.3	Initialization	3
5.4	Forward Pass	3
5.4.1	Activation Functions . . .	3
5.5	Back propagation	3
5.6	Updating Parameters	3
5.7	Non-Essential Features	3
5.7.1	Early Stopping	3
5.7.2	Mini-Batch Gradient De- scent	3
5.7.3	Weight Decay	3
6	M3 - Ensemble	3
7	Limitations	3
8	Conclusion	3
8.1	Improvements	3

1 Introduction

This paper aims to explore methods to model the claims risk of insurance policies using various machine learning methods as well as discuss their implementation. We will explore and clean the data, then define the claims risk based on the data available in the dataset.

1.1 Machine Learning Methods

In this paper we will use custom implementation of a Feed-Forward Neural Network Regressor and custom implementation of a Decision Tree Regressor using only Python numerical libraries, e.g. Python. Furthermore, we will use an ensemble method using a combination of **TBD**

2 Data

The dataset that we used for this report is the French Motor Third-Part Liability (`freMTPL2freq`) dataset from the `CASdatasets` (Dutang et al., 2024, p. 71) R package.

2.1 Data Format

See Table 1 for names and descriptions of the data columns.

2.2 Data Cleaning

For data cleaning we decided to take a conservative approach minimally modify the data. We concluded that the data did not have any obvious signs of subverting our assumptions, except for the following:

1. Exposure should be in range $(0, 1]$ since the data was collected over one year, so we assume that any value above one is an error, and we set it equal to the max, so 1.
2. We removed any data points where $VehAge > 30$ as this is the cutoff at which Vehicles become classified as historic cars, and are treated differently.

2.3 Visualization

yappity

3 Features

Although we said that we believe that there is not much to clean in the data, we opted not to use some aspects of it. Namely, we dropped these columns (**EDIT THESE IF CHANGES**)

Table 1: Data columns and descriptions.

Name	Description
IDpol	Unique policy ID.
ClaimNb	Number of claims made in policy.
Exposure	Duration of policy in years.
Area	Density category of driver's residence from "A" for rural area to "F" for urban centre.
VehPower	Power category of car.
VehAge	Age of car.
DrivAge	Age of driver.
BonusMalus	Bonus/malus, between 50 and 350: < 100 means bonus, > 100 means malus.
VehBrand	The car brand divided in the following groups: A- Renault Nissan and Citroen, B- Volkswagen, Audi, Skoda and Seat, C- Opel, General Motors and Ford, D- Fiat, E- Mercedes Chryslerand BMW, F- Japanese (except Nissan) and Korean, G- other.
VehGas	Type of gas that the vehicle uses.
Density	Number of inhabitants per km^2 in the city the driver of the car lives in.
Region	The policy region in France (based on the 1970-2015 classification).

- a. `IDpol` - not a feature
- b. `Area` - Categorical version of Density
- c. `VehBrand` - Vehicle brand alone does not indicate risk of policy since a singular brand has many models of vehicle, some of which might attract more risky drivers, however we believe that the risk evens out along the whole lineup of cars from a brand, well in this case group of brands.

d. Region - Another indicator of Density.

So after dropping these we are left with our feature columns: VehPower, VehAge, DrivAge, BonusMalus, VehGas, Density and the columns on which we base our Risk calculations: ClaimNb and Exposure.

3.1 Parameters

which and why, maybe use itemize and make it as a list

3.2 Clustering

yuh yuh yap

3.3 PCA

yap yap yap

4 M1 - Decision Tree

5 M2 - Feed-Forward Neural Network

A Feed-Forward Neural Network (*FFNN*) is our second custom implemented regression method. In this section we will explain the basics of how a FFNN works along with how we implemented these methods.

5.1 The Structure

A FFNN is a collection of nodes organized in layers. First, the input layer. The input layer number of nodes is equal to the number of features in the dataset. Second, the hidden layers. There can be an arbitrary number of hidden layers and of nodes per hidden layer. Generally, the more hidden layers or nodes per layer the more complex the Neural Network becomes, which makes it more prone to overfitting and more computationally heavy to train and run predictions on. Lastly, the output layer. The number of nodes in the output layer depends on the task, but for regression there is just one node, which give the value predicted value for the data points.

Figure 1: Structure of a FFNN

5.2 The Flow

A FFNN at its core follows a very simple flow, see Figure 2

Figure 2: Flowchart of a FFNN

5.3 Initialization

We initialize an array of weights (W_{ij}) and an array of biases (b_j) for each pair of subsequent layers. The shape of W_{ij} is the number of nodes in the previous layer (i) by the number of nodes in the current layer (j) which is also the shape of the bias array.

To fill the arrays with initial values we use the He Initialization (He et al., 2015) by drawing from a zero-mean normal distribution with standard deviation of $\sqrt{2/n_i}$ which is considered to be among the best initializations for shallow ReLU based FFNNs.

5.4 Forward Pass

During the Forward Pass, the data, as the name suggests propagates forward through the network. At each layer, except the input layer, the activations of the previous layer (a_i) are weighed by the weights and have the bias added to them:

$$z_j = a_i W_{ij} + b_j$$

Then they are passed through the activation function to become the activation of the current layer, which are then passed to the next layer.

5.4.1 Activation Functions

There are many activation functions that one might choose depending on the task. Usually you use two different activation functions, one for the hidden layers and a different one for the output layer. For hidden layers we use the widely used activation function, the Rectified Linear Unit (*ReLU*) function defined as $ReLU(x) = \max(0, x)$. For the output layer of regression FFNNs a simple linear function is used $f(x) = x$

5.5 Back propagation

5.6 Updating Parameters

5.7 Non-Essential Features

5.7.1 Early Stopping

5.7.2 Mini-Batch Gradient Descent

5.7.3 Weight Decay

6 M3 - Ensemble

7 Limitations

8 Conclusion

8.1 Improvements

braaa crkk

References

- [Dutang et al., 2024] Dutang, C., Charpentier, A., Gallic, E., and Siharath, J. (2024). CASdatasets: Insurance datasets. <https://cas.uqam.ca/pub/web/CASdatasets-manual.pdf>. R package version 1.2-0.
- [He et al., 2015] He, K., Zhang, X., Ren, S., and Sun, J. (2015). Delving deep into rectifiers: Surpassing human-level performance on imagenet classification.